



ZÁRÓDOLGOZAT

Készítették:

Almássy Benjámín – Pázmándi Milán- Szentpéteri Gellért

Konzulens:

Farkas Zoltán

Miskolc

2026.

Miskolci SZC Kandó Kálmán Informatikai Technikum

Miskolci Szakképzési Centrum

SZOFTVERFEJLESZTŐ- ÉS TESZTELŐ SZAK

DOKUMENTÁCIÓ

Faipari webshop és időpontfoglaló rendszer

Almássy Benjámin – Pázmándi Milán - Szentpéteri Gellért

2025-2026

Tartalom

1.Bevezetés.....	1
2. Felhasznált Technológiák.....	2
2.1. ASP.NET Core.....	2
2.2. Entity Framework Core.....	2
2.3. JSON Web Token (JWT) hitelesítés	2
2.4. React.....	2
2.5. OpenAPI és Swagger.....	2
3. Adatbázis Tervezése.....	3
3.1. Kezdeti Adatbázis Modell	3
3.2. Adatbázis Tervezési szempontok.....	4
3.3. A megvalósított Adatbázis Struktúra	4
3. Backend rendszer működésének bemutatása	6
3.1. Backend architektúra áttekintése	6
3.2. Hitelesítés és jogosultságkezelés	7
3.3. Termék- és kategóriakezelés.....	8
3.4. Kosár és rendeléskezelés	9
3.5. Időpontfoglalási rendszer	10
3.6. Email értesítési mechanizmus	11
3.7. Regisztráció és email megerősítés folyamata.....	12
4. Frontend rendszer bemutatása.....	13
4.1. A frontend architektúrája	13
4.2. Hitelesítés a frontend oldalon.....	14
4.3. Kosár és rendeléskezelés a frontendben	15
4.4. Időpontfoglalás a frontendben.....	16
4.5. Email megerősítés a frontendben	17
5. Felhasználói Dokumentáció.....	18
5.1. Főoldal.....	18
5.2. WEBSHOP -Termékek oldal.....	19
5.3. Termékekről oldal.....	20
5.4. Időpontfoglalás oldal	21
5.5. Regisztráció	22
5.6. Bejelentkezés.....	23
5.7. Kosár és Rendeléseim oldalak.....	24
5.8. Admin felület.....	25
5.8.1. Admin oldal	25

5.8.2. Termékek (Admin) oldal	26
5.8.3. Rendelések (Admin) oldal	27
5.8.4. Időpontfoglalások (Admin) oldal	28
6. Továbbfejlesztési lehetőségek	29
7. Összegzés	30
8. Tesztelési dokumentáció	31
8.1 Tesztelés célja	31
8.2. UI Tesztelés	31
8.3. Selenium alapú tesztelés (End-to-End)	34
9. Irodalomjegyzék	38

1.Bevezetés

A kis- és középvállalkozások működésében egyre nagyobb szerepet kapnak a digitális megoldások, amelyek segítik a mindennapi adminisztrációt, az ügyfélkapcsolatok kezelését és az értékesítési folyamatok átláthatóbbá tételét. Egy jól megtervezett webes rendszer nemcsak az online jelenlétet erősíti, hanem hatékonyabbá teheti a vállalkozás belső folyamatait is.

A dolgozatban bemutatott projekt egy valós igény alapján készült. A fejlesztés ötlete egy olyan kisvállalkozás működéséből indult ki, amelyet a szomszédom és egyben gyerekkori jó barátom működtet. A vállalkozás elsősorban jó minőségű, külföldről – főként Németországból és Ausztriából – származó faépítő csavarokat, építőipari fóliákat, valamint egyéb korszerű építőipari technológiákat és kiegészítő termékeket értékesít.

A vállalkozás működésének sajátossága, hogy az értékesítés jelentős része nem hagyományos webáruházon keresztül történik, hanem közvetlen kapcsolatokon, személyes ajánlásokon és visszatérő ügyfeleken alapuló direkt értékesítés formájában. Emiatt korábban nem volt szükség egy teljes funkcionalitású online értékesítési rendszerre, ugyanakkor egy informatikai megoldás jelentősen megkönnyítheti a termékek kezelését, a rendelések nyomon követését és az ügyfelekkel való kapcsolattartást.

A projekt célja egy olyan webes alkalmazás létrehozása volt, amely a vállalkozás működéséhez illeszkedő alapvető funkciókat biztosít. A rendszer lehetőséget ad a felhasználók regisztrációjára és azonosítására, a termékek megjelenítésére és kezelésére, valamint a rendelések nyomon követésére. Emellett a rendszer tartalmaz egy időpontfoglalási modult is, amely segítségével az ügyfelek előre egyeztethetnek személyes találkozókat vagy konzultációkat. Ez különösen fontos a vállalkozás működésében, mivel a termékek értékesítése sok esetben személyes egyeztetést vagy helyszíni megbeszélést igényel.

A rendszer adminisztrációs felületet is biztosít, amelyen keresztül a vállalkozás kezelni tudja a termékeket, a megrendeléseket és a foglalásokat. A fejlesztés során fontos szempont volt a rendszer átlátható felépítése, a biztonságos felhasználókezelés és a modern webfejlesztési technológiák alkalmazása.

A dolgozat célja a rendszer tervezésének és megvalósításának bemutatása, különös tekintettel a backend és frontend architektúrára, az alkalmazott technológiákra, valamint a rendszer főbb funkcionális moduljaira.

2. Felhasznált Technológiák

2.1. ASP.NET Core

A projekt backend része ASP.NET Core keretrendszerrel készült. Az ASP.NET Core egy nyílt forráskódú webalkalmazás-fejlesztési keretrendszer, amelyet a Microsoft fejleszt. Segítségével webes alkalmazások és REST API-k hozhatók létre. A keretrendszer támogatja a többplatformos működést, így Windows és Linux rendszereken is futtatható [1].

2.2. Entity Framework Core

Az adatbázis kezeléséhez az Entity Framework Core keretrendszert használja az alkalmazás. Ez egy objektum-relációs leképező (ORM) technológia, amely lehetővé teszi, hogy az adatbázissal való kommunikáció objektumokon keresztül történjen. Ez leegyszerűsíti az adatbázis-műveletek megvalósítását és csökkenti a közvetlen SQL lekérdezések számát [2].

2.3. JSON Web Token (JWT) hitelesítés

A felhasználók azonosítására JSON Web Token (JWT) alapú hitelesítést alkalmaz a rendszer. A JWT egy szabványos tokenformátum, amely lehetővé teszi a felhasználók biztonságos azonosítását. A token tartalmazza a felhasználó azonosításához szükséges adatokat, és minden kérés során elküldésre kerül a szerver felé [3].

2.4. React

A felhasználói felület React JavaScript könyvtár segítségével készült. A React komponens alapú megközelítést használ, amely lehetővé teszi a felhasználói felület kisebb, újrafelhasználható elemekből történő felépítését. A technológia különösen alkalmas dinamikus webalkalmazások fejlesztésére [4].

2.5. OpenAPI és Swagger

Az alkalmazás API végpontjainak dokumentálásához OpenAPI specifikációt és Swagger eszközt használtunk. Ezek segítségével a backend API végpontjai könnyen dokumentálhatók és tesztelhetők egy webes felületen keresztül [5][6].

3. Adatbázis Tervezése

A rendszer működéséhez relációs adatbázis került kialakításra. Az adatbázis feladata a felhasználók, a termékek, a rendelések és az időpontfoglalások adatainak tárolása. A tervezés során fontos szempont volt az adatok átlátható szerkezete és a későbbi bővíthetőség.

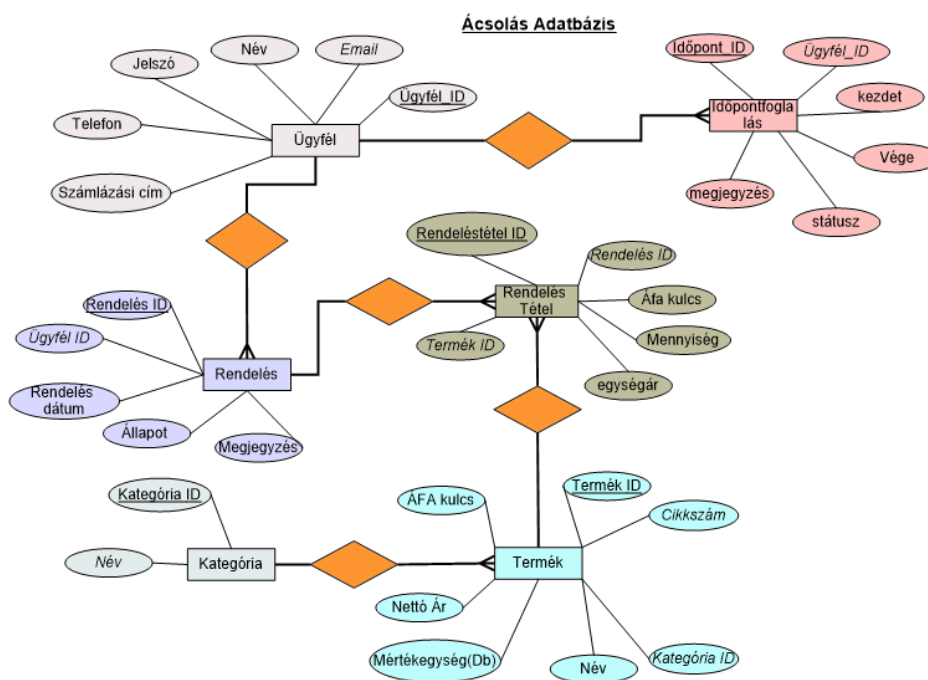
3.1. Kezdeti Adatbázis Modell

A fejlesztés elején egy entitás–kapcsolat (ER) diagram készült, amely bemutatja a rendszer főbb adatobjektumait és azok kapcsolatait.

A modell legfontosabb elemei:

- **Ügyfél** – a felhasználók adatait tartalmazza (név, email, jelszó, telefonszám, számlázási cím)
- **Rendelés** – a leadott rendelések adatait tárolja
- **Rendelés tétel** – az egyes rendelésekhez tartozó termékeket tartalmazza
- **Termék** – a webshopban megjelenő termékek adatai
- **Kategória** – a termékek csoportosítására szolgál
- **Időpontfoglalás** – a felhasználók által lefoglalt időpontokat tartalmazza

Az ER diagram a rendszer alapvető adatkapcsolatait szemlélteti.



1. ábra – Az adatbázis kezdeti logikai modellje

3.2. Adatbázis Tervezési szempontok

Az adatbázis kialakítása során fontos volt, hogy az adatok ne ismétlődjenek feleslegesen, és a rendszer könnyen bővíthető maradjon.

A **rendelések** és a **rendelési tételek** külön táblában szerepelnek. Erre azért volt szükség, mert egy rendelés több terméket is tartalmazhat. Az **order** tábla a rendelés alapadatait tárolja, míg az **order_item** tábla az adott rendeléshez tartozó termékeket és azok mennyiségét.

A **kategóriák** külön táblában találhatók, mert több termék is tartozhat ugyanahhoz a kategóriához. Így a kategória nevét csak egyszer kell tárolni, a termékek pedig idegen kulccsal hivatkoznak rá.

Az **időpontfoglalások** szintén külön táblába kerültek, mivel egy felhasználó több időpontot is foglalhat. A táblában a foglalás kezdési és befejezési ideje, valamint a státusza is tárolásra kerül.

3.3. A megvalósított Adatbázis Struktúra

A fejlesztés során az adatbázis szerkezete tovább bővült, hogy minden szükséges funkciót kiszolgáljon. A végleges adatbázis több egymással kapcsolatban álló táblából épül fel.

A legfontosabb táblák a következők:

client

A felhasználók adatait tárolja, például a nevet, email címet, telefonszámot, számlázási címet, valamint a jelszó hash értékét és a felhasználói jogosultságot.

product

A webshopban szereplő termékek adatait tartalmazza, például a cikkszámot, a termék nevét, kategóriáját, nettó árát és mértékegységét.

category

A termékek kategóriáit tárolja.

order

A felhasználók által leadott rendelések alapadatait tartalmazza.

order_item

Az egyes rendelésekhez tartozó terméktételeket tárolja.

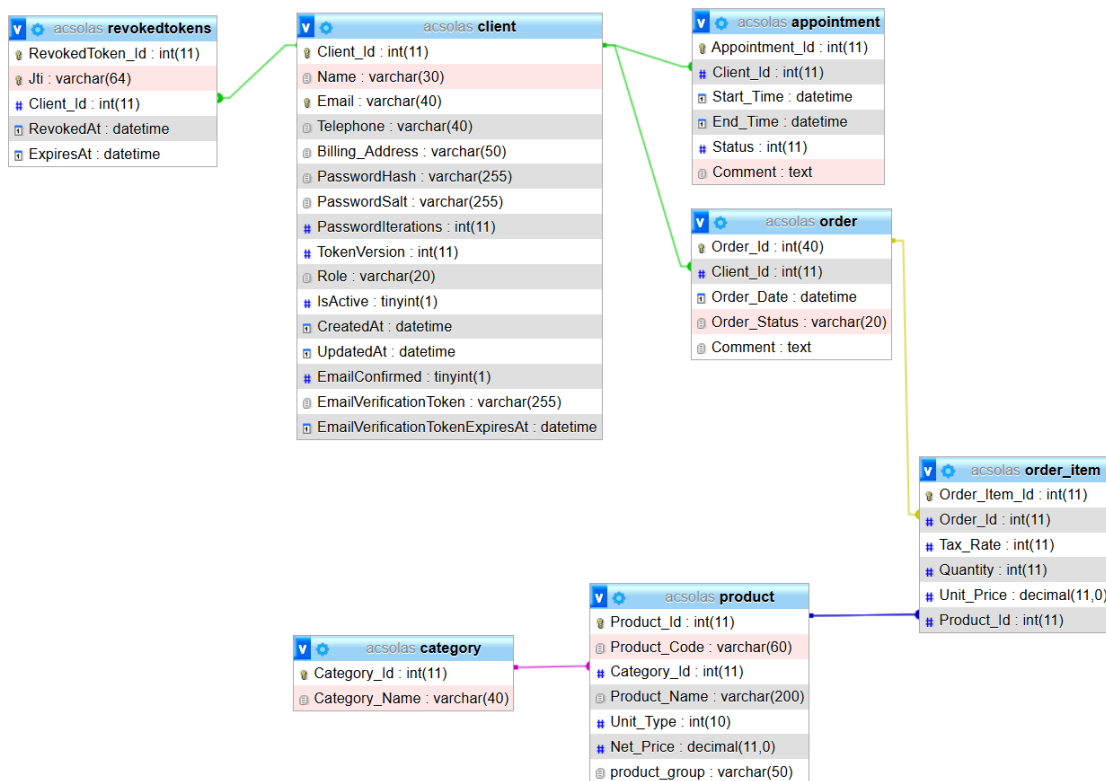
appointment

Az időpontfoglalási rendszerben rögzített foglalásokat tartalmazza.

revokedtokens

A visszavont hitelesítési tokenek tárolására szolgál.

A táblák idegen kulcsok segítségével kapcsolódnak egymáshoz, így biztosítva az adatok közötti kapcsolatokat.



2. ábra – A rendszer végleges adatbázis struktúrája

A kialakított adatbázis-szerkezet biztosítja a rendszer működéséhez szükséges adatok strukturált tárolását, valamint támogatja a webshop és az időpontfoglalási rendszer funkcióinak megbízható működését.

3. Backend rendszer működésének bemutatása

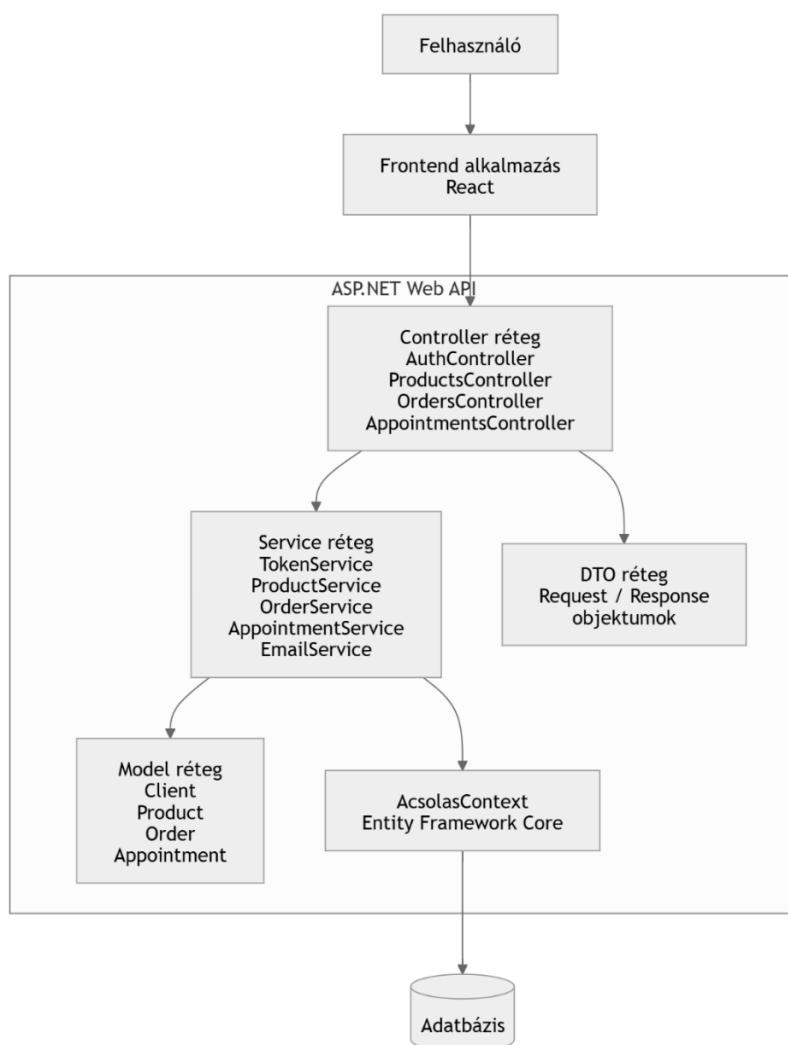
3.1. Backend architektúra áttekintése

A backend alkalmazás réteges architektúrát követ, amely elkülöníti a kliens kommunikációt, az üzleti logikát és az adatelérést. Ez a struktúra javítja a rendszer karbantarthatóságát, bővíthetőségét és átláthatóságát.

A kliens oldali alkalmazás HTTP kéréseken keresztül kommunikál a szerverrel. A kérések a controller réteghez érkeznek, ahol megtörténik a bemeneti adatok feldolgozása és validálása. A kontrollerek az üzleti logika végrehajtását a service rétegnek adják át.

A service réteg valósítja meg a rendszer fő működését, például a hitelesítést, a rendeléskezelést vagy az időpontfoglalást. Az adatok kezelése az Entity Framework Core segítségével történik, amely az AcsolasContext DbContext osztályon keresztül biztosít kapcsolatot az adatbázissal.

A rendszer modelljei reprezentálják az adatbázis tábláit és azok kapcsolatait.



3. ábra – A rendszer architektúrájának áttekintése

3.2. Hitelesítés és jogosultságkezelés

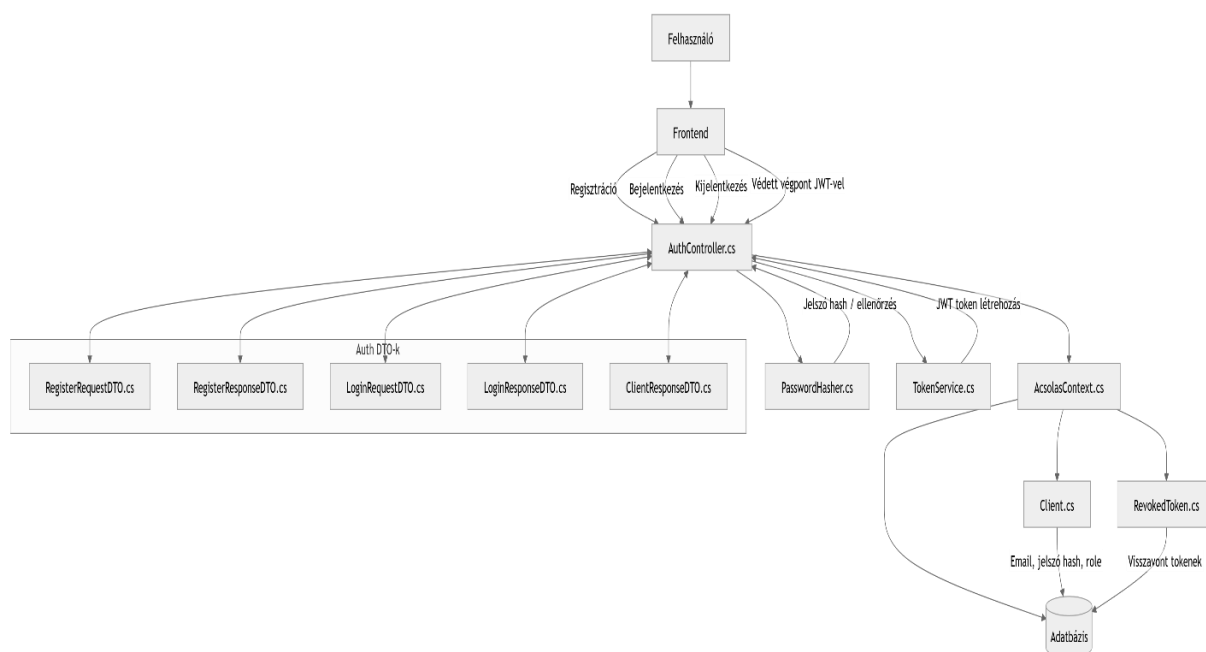
A rendszer hitelesítése JWT (JSON Web Token) alapú mechanizmust használ. A hitelesítési folyamat az AuthController vezérlésével történik.

Regisztráció során a felhasználó megadja az adatait, amelyeket a rendszer validál. A jelszó biztonsági okokból nem kerül közvetlenül tárolásra, hanem hash formában mentésre kerül a PasswordHasher komponens segítségével.

Bejelentkezéskor a rendszer ellenőrzi a megadott email címet és jelszót. Ha az adatok helyesek, a TokenService egy JWT tokenet generál, amely tartalmazza a felhasználó azonosítóját és szerepkörét. A kliens ezt a tokenet használja a későbbi kérések hitelesítésére.

A védett végpontok csak érvényes token birtokában érhetők el. A rendszer role-alapú jogosultságkezelést is alkalmaz, így bizonyos műveletek csak adminisztrátori jogosultsággal hajthatók végre.

Kijelentkezés esetén a token azonosítója a visszavont tokenek listájába kerül, amely megakadályozza annak további használatát.



4. ábra – A hitelesítési folyamat működése

3.3. Termék- és kategóriakezelés

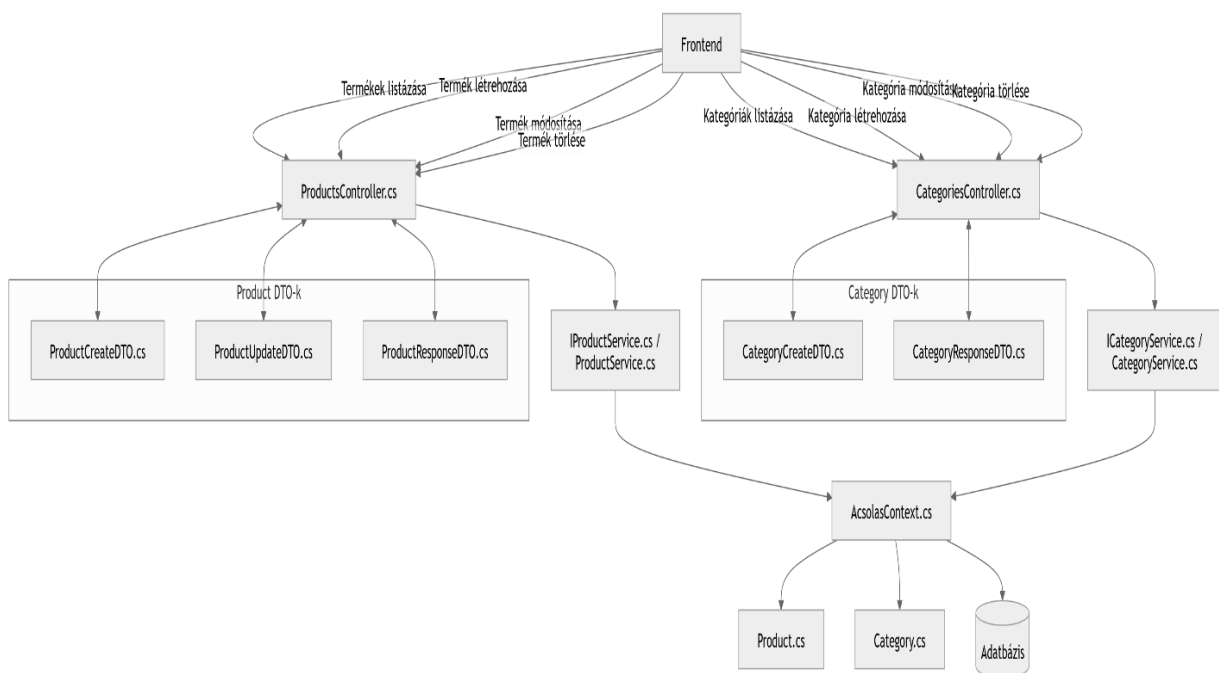
A termékek és kategóriák kezelését külön kontrollerek és service osztályok végzik.

A ProductsController felelős a termékek listázásáért, létrehozásáért, módosításáért és törléséért. A CategoriesController hasonló módon kezeli a kategóriákat.

A kontrollerek DTO objektumokat használnak a bemeneti és kimeneti adatok kezelésére. Ez lehetővé teszi, hogy az API ne közvetlenül a modellel dolgozzon, így a rendszer rugalmasabban módosítható.

Az üzleti logika a ProductService és CategoryService osztályokban található. Ezek a szolgáltatások az AcsolasContext segítségével végzik az adatbázis műveleteket.

A termékek kategóriákhoz kapcsolódnak, így egy kategóriához több termék is tartozhat.



5. ábra – A termék- és kategóriakezelés architektúrája

3.4. Kosár és rendeléskezelés

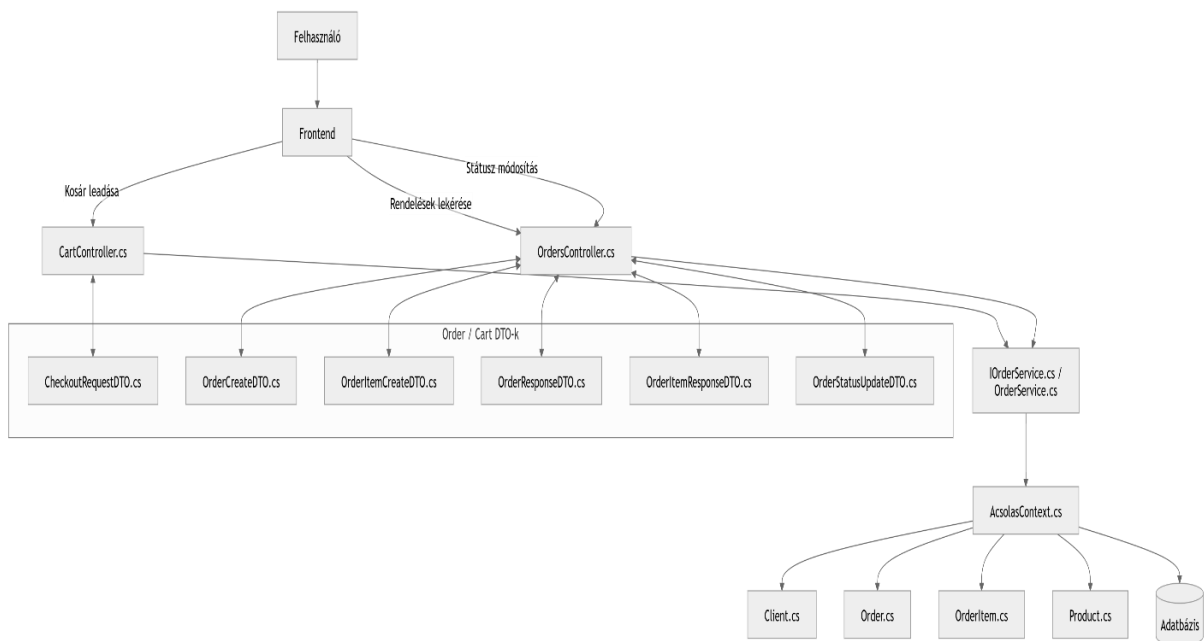
A kosár és rendelési folyamat a CartController és az OrdersController együttműködésével valósul meg.

A felhasználó a frontend felületen keresztül kiválasztja a kívánt termékeket, amelyeket a kosárba helyez. A kosár véglegesítésekor a rendszer egy checkout műveletet hajt végre.

A checkout során létrejön egy rendelés (Order) és a hozzá tartozó rendelési tételek (OrderItem). A rendelési tételek tartalmazzák a termékeket, azok mennyiségét és az aktuális árakat.

A felhasználó lekérdezheti saját rendeléseit, míg az adminisztrátor jogosultság alapján módosíthatja azok státuszát. A rendelés státusza például lehet folyamatban, jóváhagyva, teljesítve vagy törölve.

A rendelési adatok az adatbázisban az Order és OrderItem modelleken keresztül kerülnek tárolásra.



6. ábra – A rendeléskezelési folyamat architektúrája

3.5. Időpontfoglalási rendszer

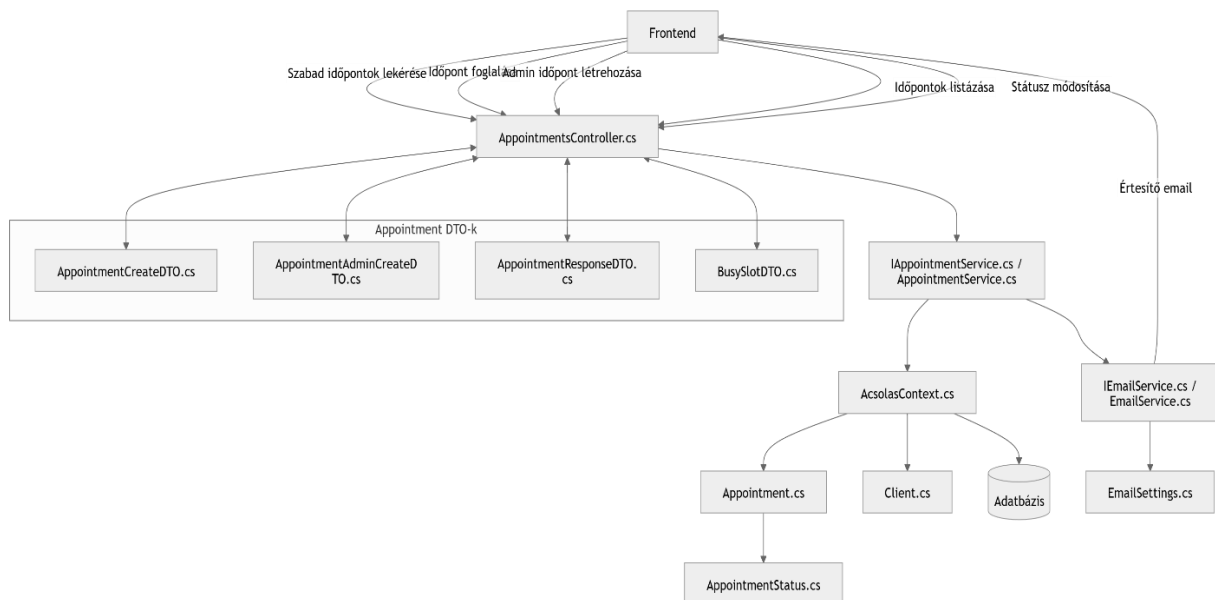
A rendszer tartalmaz egy időpontfoglalási modult is, amely az `AppointmentsController` és az `AppointmentService` segítségével működik.

A felhasználók lekérdezhetik a rendelkezésre álló időpontokat, majd kiválaszthatják a számukra megfelelő időszávot. Az új foglalás létrehozásakor a rendszer ellenőrzi, hogy az adott időpont még szabad-e.

Az adminisztrátorok jogosultság alapján új időpontokat hozhatnak létre, módosíthatják a foglalások státuszát vagy törölhetik azokat.

Az időpontok státuszát egy enumeráció írja le, amely különböző állapotokat reprezentál, például folyamatban lévő, jóváhagyott vagy törölt foglalás.

A foglalások az `Appointment` modellben kerülnek tárolásra.



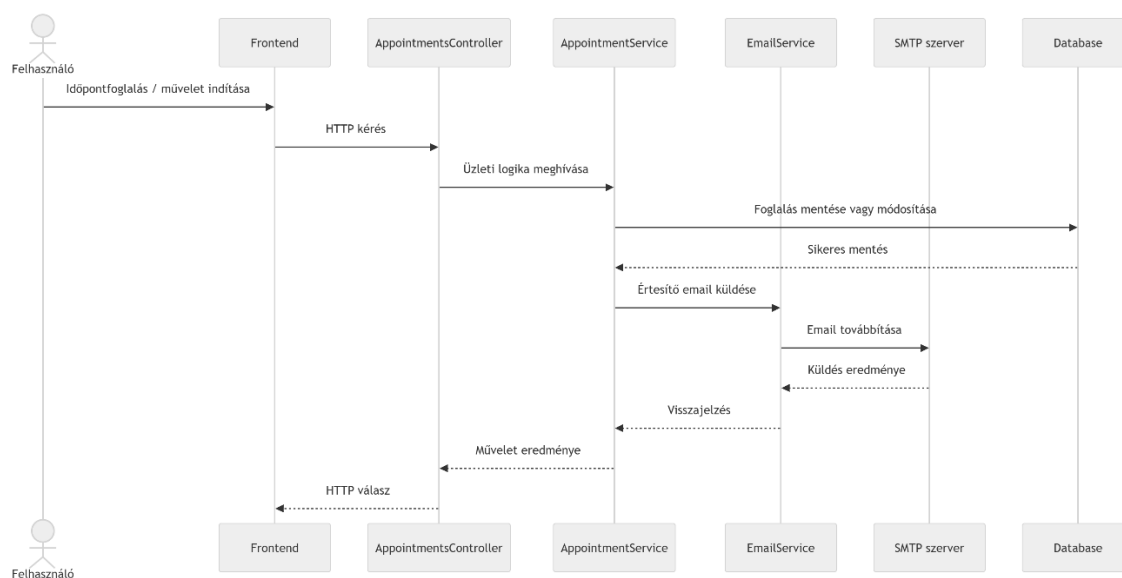
7. ábra – Az időpontfoglalási modul architektúrája

3.6. Email értesítési mechanizmus

A rendszer email értesítéseket is küld bizonyos műveletek végrehajtásakor. Az email küldés az EmailService komponens segítségével történik.

Az email küldés SMTP kapcsolaton keresztül valósul meg. A szükséges konfigurációs adatok, például a szerver címe, portja és hitelesítési adatai az EmailSettings konfigurációs osztályban találhatók.

Az email értesítések például időpontfoglalás, rendelés vagy azok módosítása esetén kerülnek kiküldésre.



8. ábra – Az időpontfoglalás folyamatának szekvenciadiagramja

3.7. Regisztráció és email megerősítés folyamata

A rendszer a felhasználói fiókok létrehozásakor email megerősítési mechanizmust alkalmaz annak érdekében, hogy biztosítsa a megadott email cím valódiságát. Ez a folyamat segít megelőzni a hamis vagy nem létező email címek használatát, valamint növeli a rendszer biztonságát.

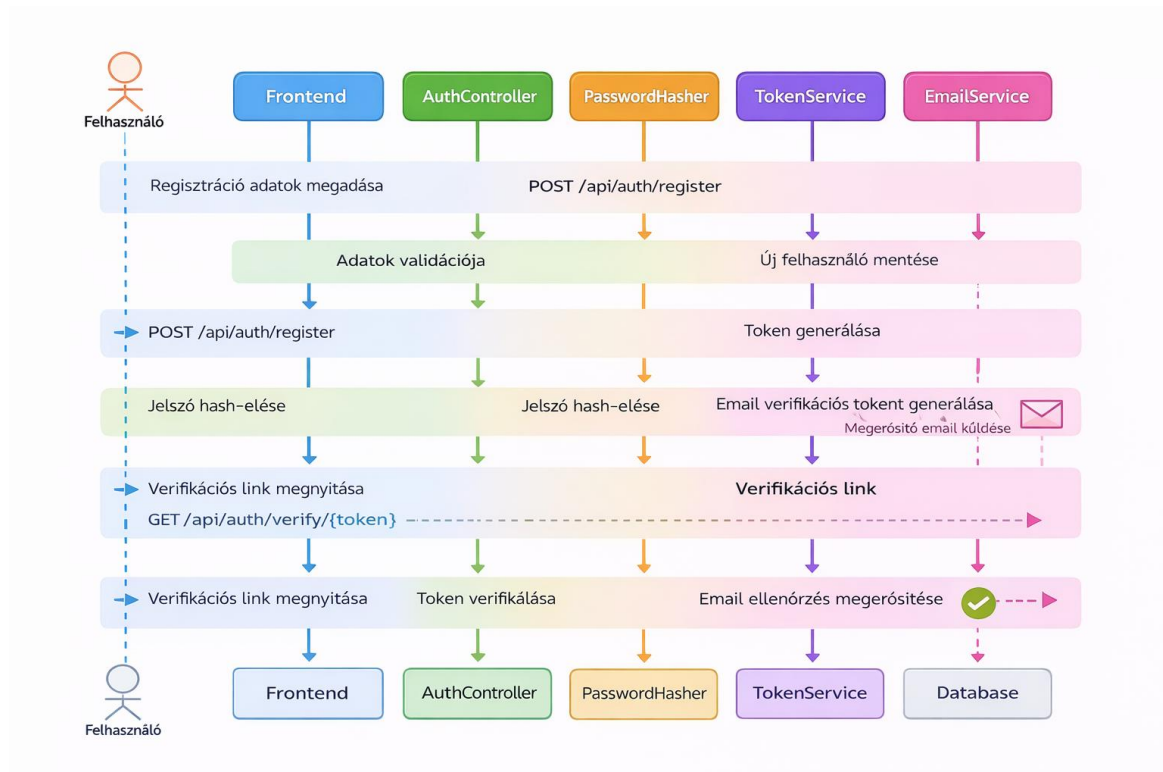
A regisztráció során a felhasználó megadja a szükséges adatokat, például a nevét, email címét és jelszavát. A backend ellenőrzi az adatok helyességét, majd létrehozza az új felhasználói rekordot az adatbázisban. A jelszó biztonsági okokból nem kerül közvetlenül tárolásra, hanem hash formában mentésre kerül.

A regisztráció sikeres végrehajtása után a rendszer generál egy egyedi azonosító tokent, amely az email megerősítéshez szükséges. Ez a token egy speciális URL részeként kerül elküldésre a felhasználó email címére.

Az email tartalmaz egy megerősítő linket, amelyre kattintva a felhasználó visszatér az alkalmazásba. A link egy token paramétert tartalmaz, amelyet a backend ellenőriz.

Amikor a felhasználó megnyitja a megerősítő linket, a frontend elküldi a tokent a backend megfelelő végpontjára. A rendszer ellenőrzi a token érvényességét, majd sikeres ellenőrzés esetén aktiválja a felhasználói fiókot.

A megerősítés után a felhasználó teljes hozzáférést kap a rendszer funkcióihoz, és bejelentkezhet az alkalmazásba.



9. ábra – A regisztráció és email megerősítés folyamatának áramlásdiagramja

4. Frontend rendszer bemutatása

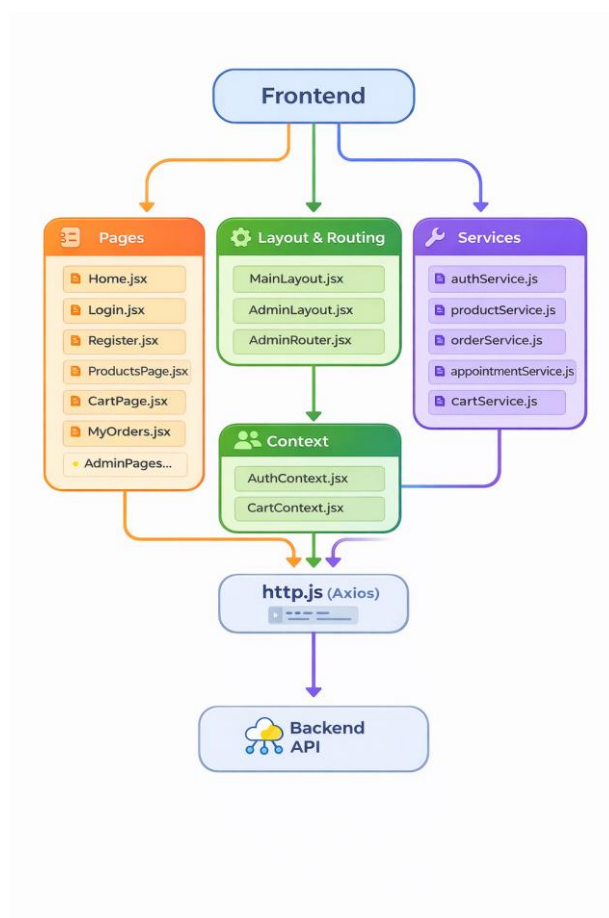
4.1. A frontend architektúrája

A projekt frontend része egy React alapú kliensalkalmazás, amely a felhasználói felület megjelenítéséért és a backend rendszerrel történő kommunikációért felelős. Az alkalmazás moduláris felépítésű, amely lehetővé teszi a különböző funkcionális részek elkülönítését és könnyebb karbantarthatóságát.

A frontend feladata, hogy a felhasználói interakciókat kezelje, megjelenítse az adatokat, valamint HTTP kéréseken keresztül kommunikáljon a backend API-val. Az alkalmazás több különálló rétegre bontható, amelyek eltérő szerepet töltenek be a rendszer működésében.

A felhasználói felületet React komponensek és oldalak alkotják. Ezek a komponensek szükség esetén service függvényeket hívnak meg, amelyek a backend API-val kommunikálnak. Az alkalmazás bizonyos globális adatait, például a felhasználó bejelentkezési állapotát vagy a kosár tartalmát context alapú állapotkezelés biztosítja.

A frontend szerkezete így jól elkülöníti a megjelenítési logikát, az alkalmazás állapotkezelését és a backend kommunikációt.



10. ábra – A frontend alkalmazás architektúrája

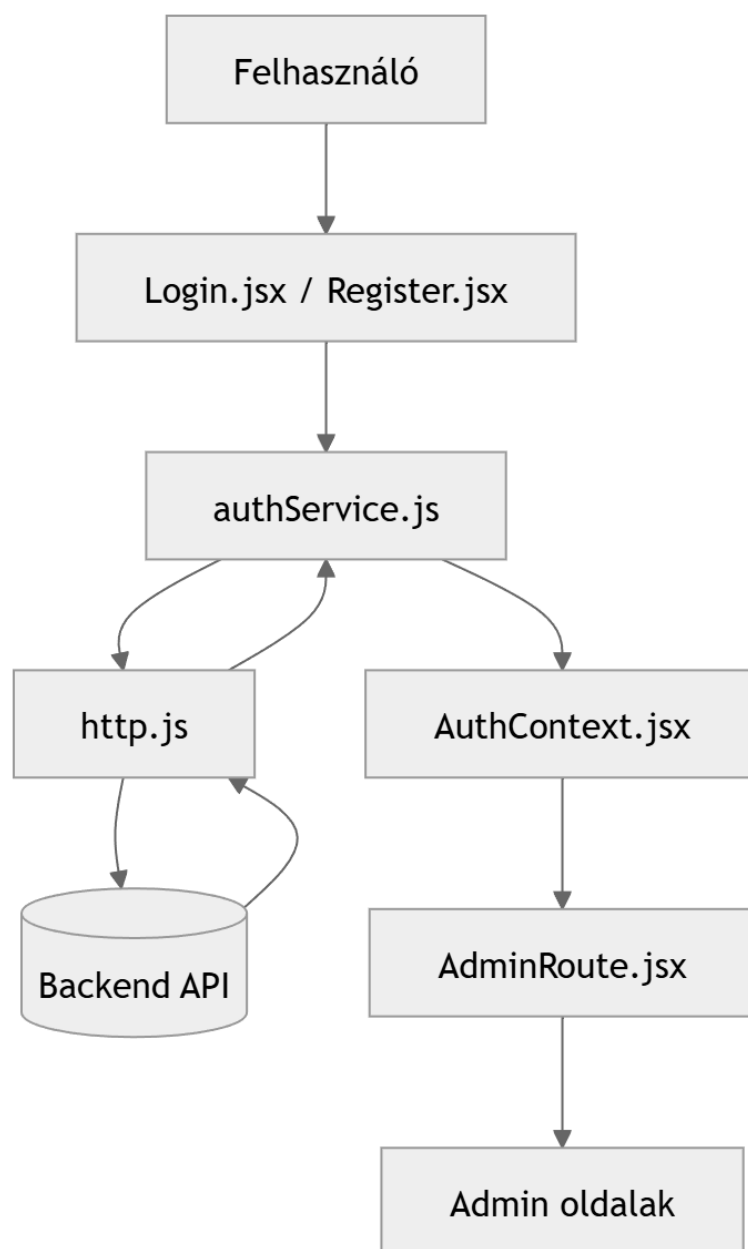
4.2. Hitelesítés a frontend oldalon

A frontend alkalmazás a felhasználók hitelesítését a `authService.js` és az `AuthContext.jsx` segítségével kezeli.

A bejelentkezési folyamat során a felhasználó megadja az email címét és jelszavát a `Login.jsx` oldalon. Ezeket az adatokat az `authService` továbbítja a backend megfelelő végpontjára.

Sikeres bejelentkezés esetén a backend JWT tokenet küld vissza. A frontend ezt a tokenet eltárolja, majd az `AuthContext` segítségével globálisan elérhetővé teszi az alkalmazás számára.

Az adminisztrátori jogosultság ellenőrzése az `AdminRoute` komponensben történik, amely eldönti, hogy a felhasználó hozzáférhet-e az adminisztrációs oldalakhoz.



11. ábra – A frontend hitelesítési és jogosultságkezelési folyamata

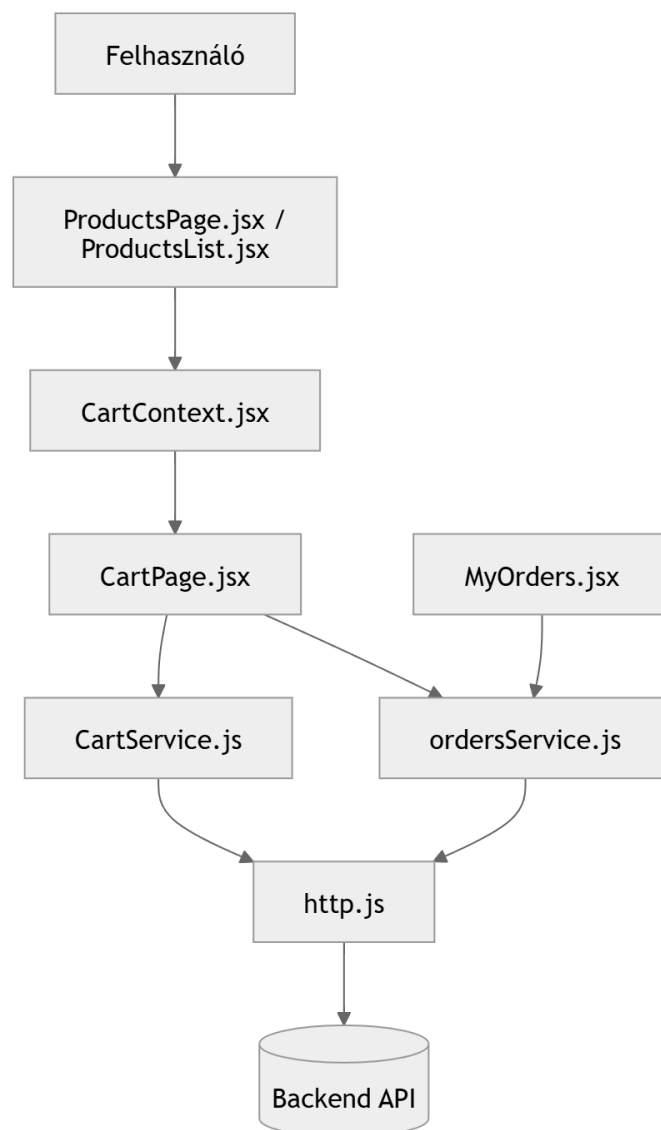
4.3. Kosár és rendeléskezelés a frontendben

A kosár kezelését a CartContext valósítja meg. Ez a context tárolja a kiválasztott termékeket és azok mennyiségét.

Amikor a felhasználó terméket ad a kosárhoz, az adat a CartContext állapotában kerül eltárolásra. Ez lehetővé teszi, hogy a kosár tartalma az alkalmazás több különböző oldalán is elérhető legyen.

A rendelés leadása a kosár oldalán történik, ahol a CartService és az ordersService segítségével a rendszer elküldi a szükséges adatokat a backendnek.

A felhasználó a MyOrders oldalon tekintheti meg korábbi rendeléseit, míg az adminisztrátor külön admin felületen kezelheti az összes rendelést.



12. ábra – A kosár és rendeléskezelés frontend folyamata

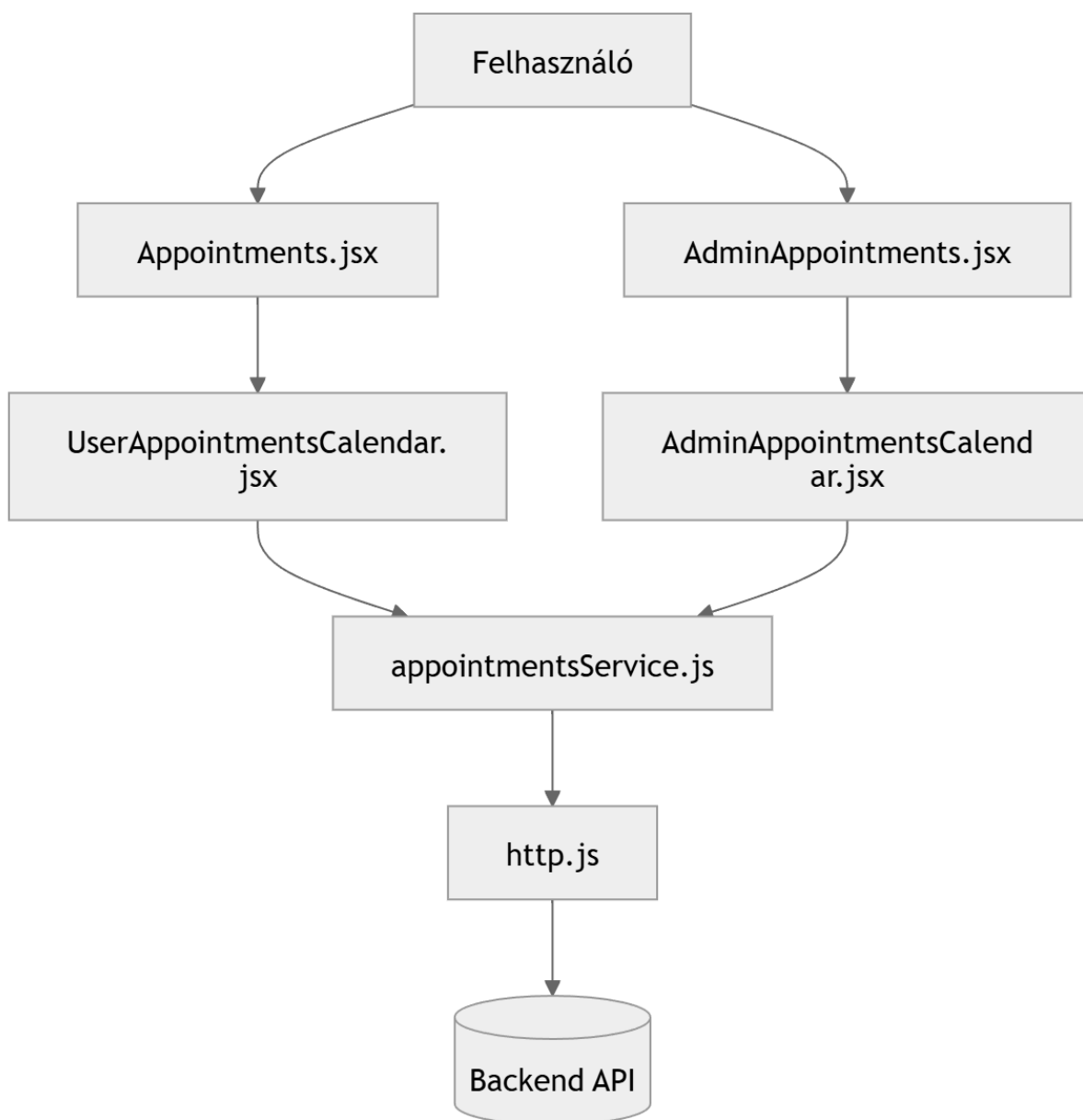
4.4. Időpontfoglalás a frontendben

Az időpontfoglalási funkció egy külön oldalon valósul meg, amely a naptár alapú időpontkezelést biztosítja.

A felhasználók a `Appointments.jsx` oldalon foglalhatnak időpontot. A foglalások kezeléséhez a `UserAppointmentsCalendar` komponens biztosítja a naptár felületet.

Az adminisztrátorok számára külön adminisztrációs felület áll rendelkezésre, ahol az időpontok létrehozhatók, módosíthatók vagy törölhetők. Ehhez az `AdminAppointmentsCalendar` komponens használható.

Az adatlekérések és módosítások az `appointmentsService` segítségével történnek.



13. ábra – A felhasználói és admin időpontkezelés frontend architektúrája

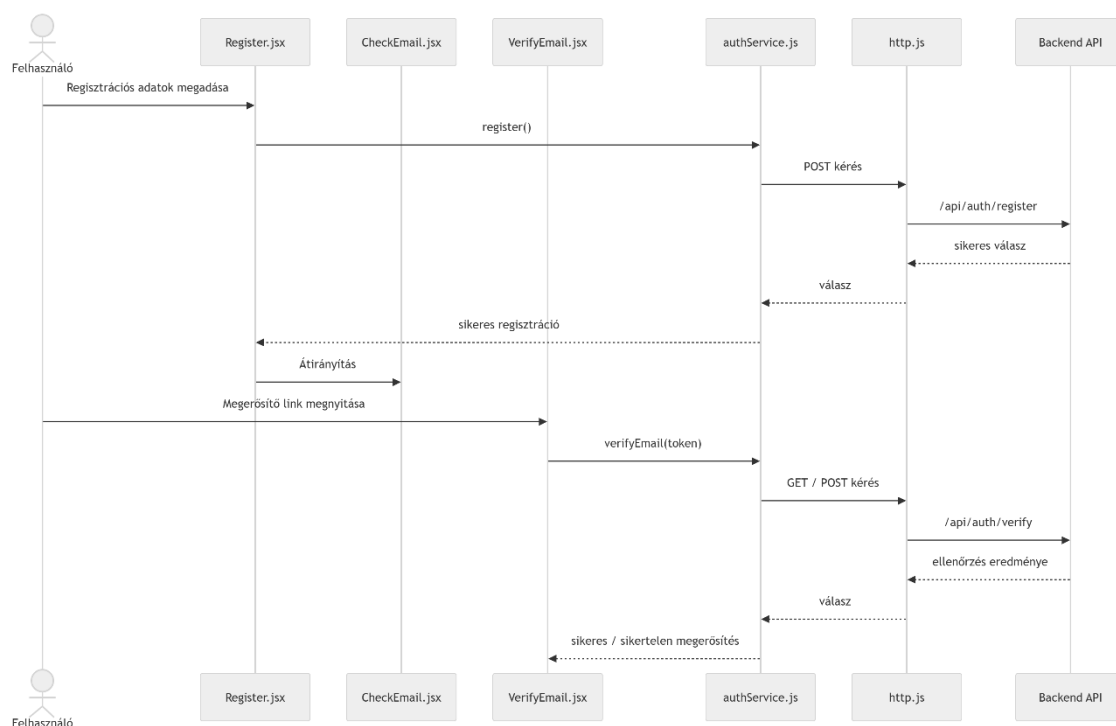
4.5. Email megerősítés a frontendben

A regisztrációt követően a felhasználó email megerősítési folyamaton megy keresztül.

A regisztráció után a rendszer a CheckEmail.jsx oldalon tájékoztatja a felhasználót arról, hogy a megerősítő email elküldésre került.

Az email egy speciális linket tartalmaz, amely egy token is magában foglal. A felhasználó erre a linkre kattintva a VerifyEmail.jsx oldalra kerül.

Ez az oldal kiolvassa a token paramétert az URL-ből, majd az authService segítségével elküldi azt a backend ellenőrzésére. Sikeres ellenőrzés esetén a rendszer megerősíti az email címet, és a felhasználó bejelentkezhet az alkalmazásba.



14. ábra – A frontend regisztráció és email verifikáció folyamata

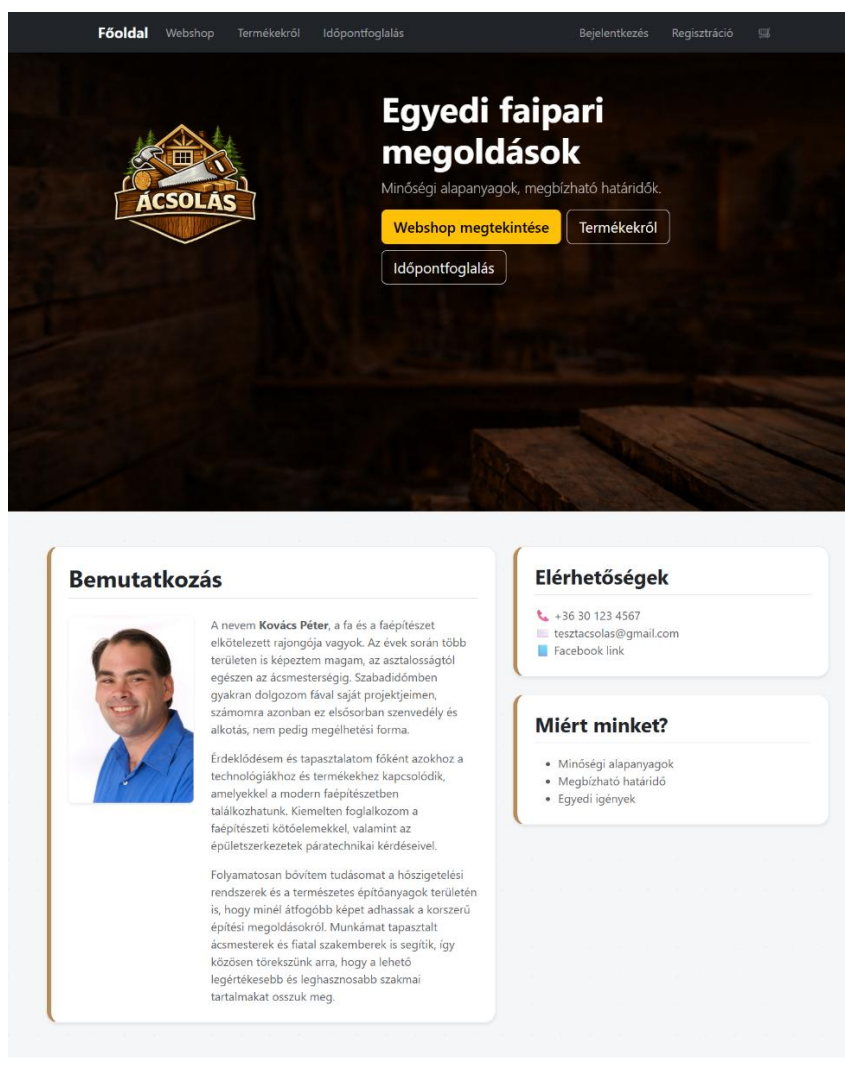
5. Felhasználói Dokumentáció

5.1. Főoldal

A főoldal a webalkalmazás kiindulópontja, ahol a felhasználó gyors áttekintést kap a szolgáltatásról és a legfontosabb funkciókról. A felső navigációs sáv segítségével a felhasználó könnyen elérheti a webshopot, a termékekről szóló információkat, valamint az időpontfoglalási felületet. Ugyanitt található a bejelentkezés és regisztráció lehetősége is.

A főoldal központi részén a rendszer röviden bemutatja a szolgáltatás jellegét, valamint gyors elérési gombokat biztosít a webshop megtekintéséhez, a termékek részletes bemutatásához és az időpontfoglaláshoz.

Az oldal alsó részén bemutatkozó szöveg található a szolgáltatóról, valamint az elérhetőségek és a szolgáltatás fő előnyei is megjelennek. Ez segíti a felhasználót abban, hogy megismerje a vállalkozást és könnyen kapcsolatba léphessen vele.



© 2026 Ácsolás – Vizsgaprojekt

15. ábra – A webalkalmazás főoldala

5.2. WEBSHOP -Termékek oldal

A termékek oldalon a felhasználó áttekintheti a webshopban elérhető termékeket. Minden termék külön kártyán jelenik meg képpel és megnevezéssel. A felhasználó a legördülő listából kiválaszthatja a kívánt terméket, majd a „Kosárba” gomb segítségével hozzáadhatja azt a bevásárlókosarához.

[Főoldal](#) [Webshop](#) [Termékekről](#) [Időpontfoglalás](#) [Bejelentkezés](#) [Regisztráció](#) 

Termékek



AEROSANA VISCONN fibre
folyékony tömítőanyag
(FEHÉR)

Válassz terméket...

Kosárba



RAPID® Süllyesztett fejű
szerkezeti facsavar

Válassz terméket...

Kosárba



Schmid RAPID® SSF és WH
Tányér fejű szerkezetépítő
csavar

Válassz terméket...

Kosárba



Pro Clima TESCON Primer RP
speciális alapozó

Válassz terméket...

Kosárba



AEROSANA VISCONN
légtömör fólia

Válassz terméket...

Kosárba



TESCON RAPIC ragasztószalag

Válassz terméket...

Kosárba



Lucfenyő gerenda 5m

Válassz terméket...

Kosárba

© 2026 Ácsolás – Vizsgaprojekt

16. ábra – A webshop terméklistázó oldala

5.3. Termékekről oldal

Az oldalon a felhasználó részletes leírást talál a kínált építőipari és faépítészeti termékekről. A bemutatott szakaszok ismertetik a termékek felhasználási területét, tulajdonságait és előnyeit. Az egyes részeknél található **„Termékek megtekintése”** gomb segítségével a felhasználó közvetlenül a webshop terméklistájára navigálhat.

[Főoldal](#) [Webshop](#) [Termékekről](#) [Időpontfoglalás](#) [Bejelentkezés](#) [Regisztráció](#)

Termékekről

Válogatott, megbízható építőipari és faépítési termékek, amelyek hozzájárulnak a tartós, energiatékony és precíz kivitelezéshez.



AEROSANA VISCONN FIBRE

Fújható, intelligens légzáró fólia

Az AEROSANA VISCONN FIBRE a pro clima rostokkal erősített, folyékony légzáró és szélzáró fóliája. Fújható anyag, amely megszáradás után rugalmas, tartós és intelligens párafékező réteget képez.

- Használható falszerkezetekben, födémeken és tetőtérben, új és régi épületeknél is.
- Különösen hasznos sarkoknál, áttöréseknél és különböző anyagok találkozásánál.
- Akár 2 cm széles repedéseket és hézagokat is áthidal segédanyagok nélkül.
- Belső és külső épületburkokban is alkalmazható lég- és szélzárásra.
- Jól tapad fához, OSB-hez, MDF-hez, betonhoz, vakolathoz, téglához, fémhez és vályoghoz is.

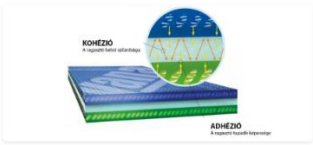
[Termékek megtekintése](#)

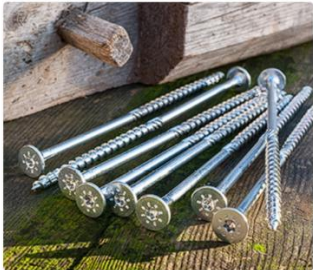
Ragasztószalagok és ragasztóanyagok

Tartós légzárás és megbízható tapadás

Az épületek hosszú távú légzárásához és szélzárásához elengedhetetlenek a jó minőségű ragasztott kötések. A megfelelő ragasztószalagok és ragasztóanyagok biztosítják az egyes szerkezeti elemek stabil és tartós összekapcsolását.

- A tartós kötés alapja a jó adhézió és a megfelelő kohézió.
- A ragasztóanyag kitölti a felület mikroszkopikus egyenetlenségeit.
- Fa, faalapú lapok, vakolat, fólia, bevonatok és műanyag felületek esetén is fontos a megfelelő választás.
- A szerkezeti mozgások felvételére is alkalmas, ha jó rendszert választunk.
- A hosszú távú légzárásnál a ragasztóanyag minősége kulcsszerepet játszik.

[Termékek megtekintése](#)



Schmid Schrauben Hainfeld

Minőségi szerkezetépítő csavar és speciális facsavar

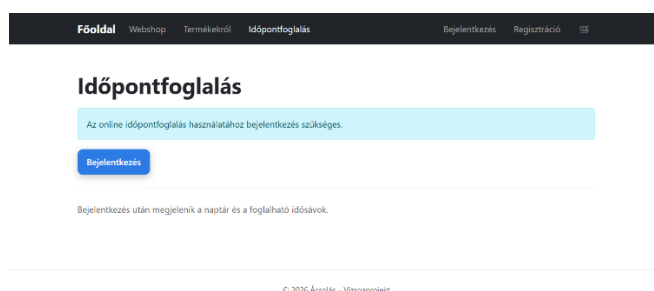
A Schmid Schrauben több mint 180 év ausztriai tapasztalattal Európa egyik vezető csavar- és rögzítéstechnikai gyártója. Saját márkáik – RAPID® és StarDrive GPR – mellett egyedül, akár 1500 mm hosszú csavarokat is gyártanak.

- Fejlett csavargeometria a gyors és precíz behajtáshoz.
- A speciális csavarhegy azonnali beharapást és gyors munkakezdést biztosít.
- A nagy menetemelkedés gyors munkavégzést tesz lehetővé.
- A dupla menetes kialakítás csökkenti a repedésveszélyt és stabil tartást ad.
- Az örlőzóna kisebb gépterhelést és hatékonyabb csavarozást eredményez.

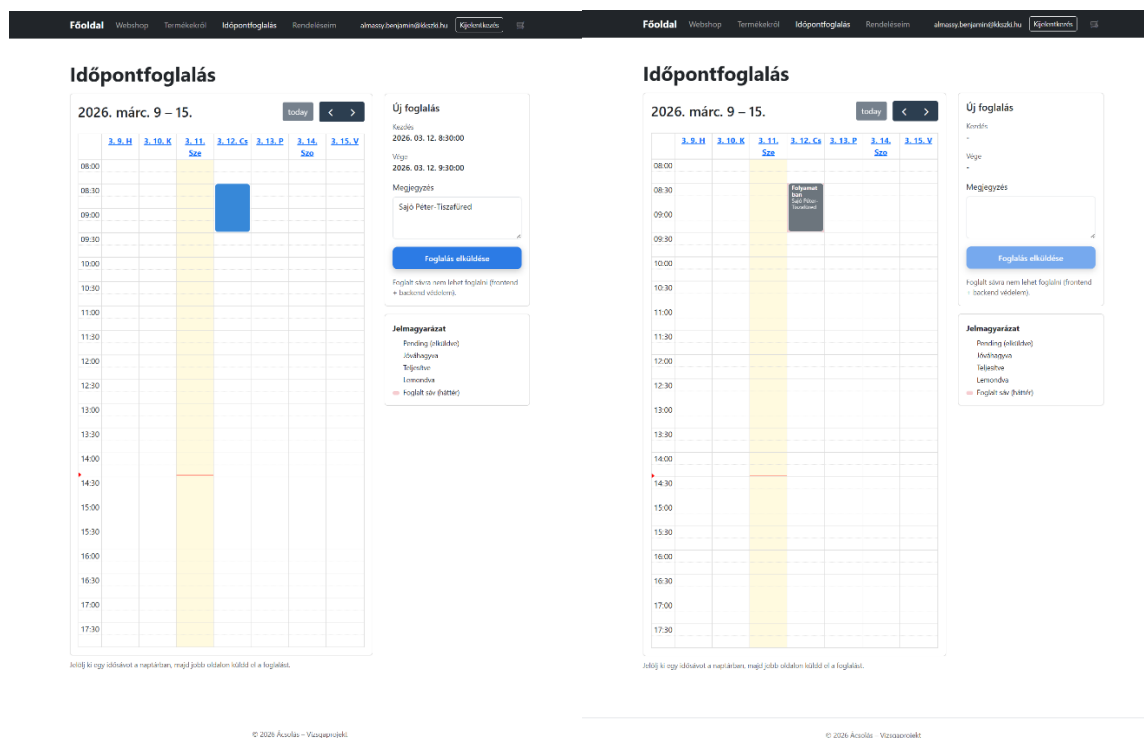
[Termékek megtekintése](#)

5.4. Időpontfoglalás oldal

Az időpontfoglalás használatához a felhasználónak be kell jelentkeznie a rendszerbe. Bejelentkezés után megjelenik a naptár, amelyben a szabad időpontok kiválaszthatók. A felhasználó a kívánt időszáv kijelölése után a jobb oldalon található űrlapon megjegyzést is megadhat, majd a **„Foglalás elküldése”** gombbal rögzítheti a foglalást.



16. ábra – Az időpontfoglalási felület kijelentkezett állapotban



18. ábra – Az időpontfoglalási felület működése bejelentkezett felhasználó esetén.

5.5. Regisztráció

A regisztráció során a felhasználó megadja a szükséges adatokat (név, email cím, telefonszám, számlázási cím és jelszó). A regisztráció elküldése után a rendszer megerősítő emailt küld a megadott címre. A felhasználó az emailben található linkre kattintva tudja megerősíteni a fiókját, ezt követően pedig bejelentkezhet az alkalmazásba.

The image displays the registration process in a web application, consisting of three main parts:

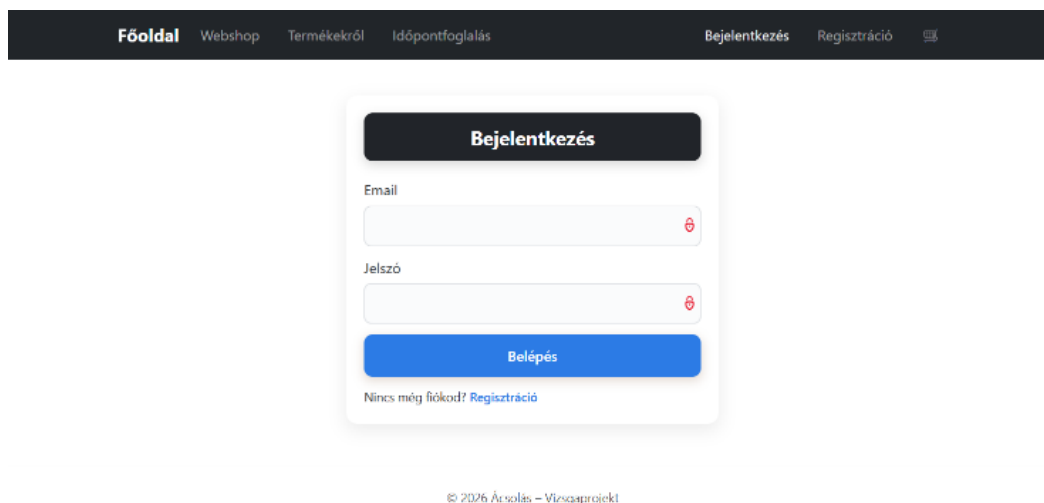
- Registration Form:** A form titled "Regisztráció" with input fields for "Név", "Email", "Telefonszám", "Számlázási cím", "Jelszó", and "Jelszó újra". A blue "Regisztráció" button is at the bottom. Below the button, it says "Van már fiókod? [Bejelentkezés](#)".
- Registration Success Message:** A message titled "Regisztráció sikeres" stating: "A fiókod létrejött. A belépés előtt meg kell erősítened az email címedet. Nézd meg a postafiókodat, és kattints a megerősítő linkre. Ha nem találod, nézd meg a spam mappát is. [Ugrás a bejelentkezéshez](#)".
- Email Verification Page:** A page titled "Email megerősítés" with the text: "Az email cím sikeresen megerősítve. Hamarosan átirányítunk a bejelentkezéshez..." and a blue link: "[Tovább a bejelentkezéshez](#)".

The footer of the application is visible in all three parts, showing the navigation menu and the copyright notice: "© 2026 Ácsolás - Vizsgaprojekt".

19. ábra – A regisztráció folyamata

5.6. Bejelentkezés

A bejelentkezési oldalon a felhasználó az email cím és a jelszó megadásával tud belépni a rendszerbe. A „**Belépés**” gombra kattintva a rendszer ellenőrzi az adatokat, és sikeres hitelesítés esetén hozzáférést biztosít a felhasználói funkciókhoz, például a rendelései megtekintéséhez és az időpontfoglaláshoz.

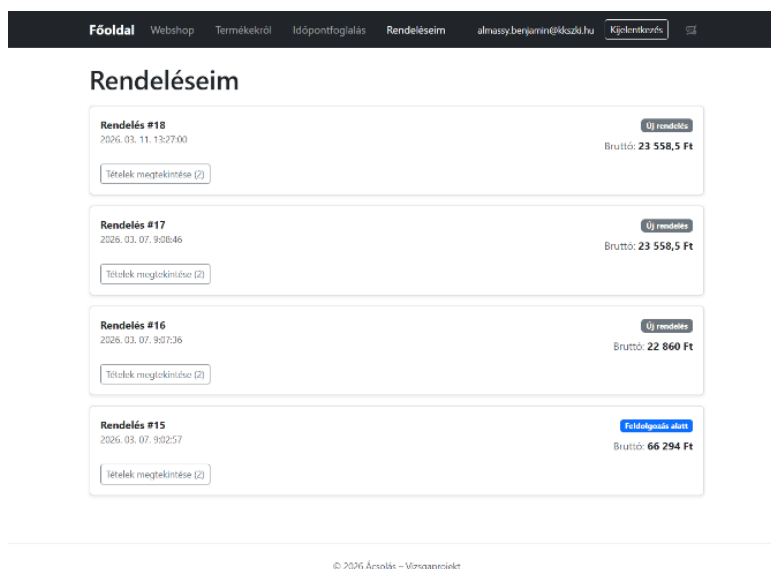
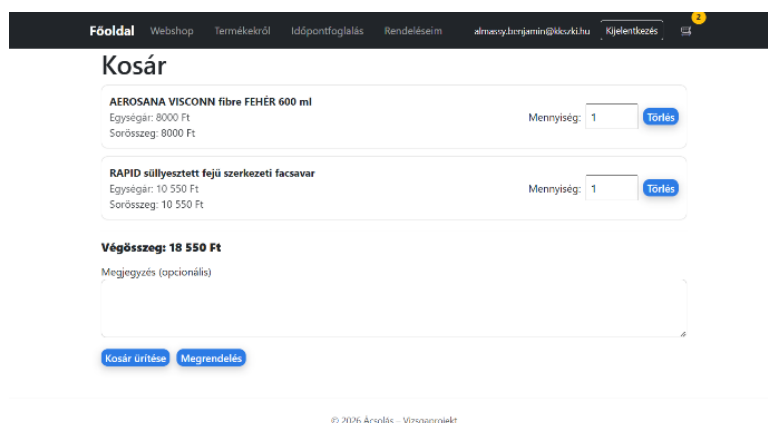


The screenshot displays a web application's login interface. At the top, a dark navigation bar contains links: Főoldal, Webshop, Termékekről, Időpontfoglalás, Bejelentkezés, and Regisztráció. The main content area features a white card titled "Bejelentkezés". Inside the card, there are two input fields: "Email" and "Jelszó", each with a red eye icon for toggling visibility. Below these fields is a blue "Belépés" button. At the bottom of the card, a link "Nincs még fiókod? Regisztráció" is provided. The footer of the page shows the copyright notice "© 2026 Ácsolás – Vízszaprojekt".

20. ábra – A bejelentkezési felület

5.7. Kosár és Rendelésem oldalak

A kosár tartalmát bárki megtekintheti, aki terméket helyezett a kosárba, azonban rendelést csak bejelentkezett felhasználó tud leadni. Amennyiben a felhasználó nincs bejelentkezve, a rendszer felugró üzenetben jelzi, hogy a rendelés leadásához be kell jelentkeznie. A leadott rendelések a „**Rendelésem**” menüpont alatt tekinthetők meg, ahol a felhasználó listában láthatja korábbi rendeléseit, azok részleteit, valamint a rendelés aktuális státuszát.



21. ábra – Kosár és Rendelésem oldalak

5.8. Admin felület

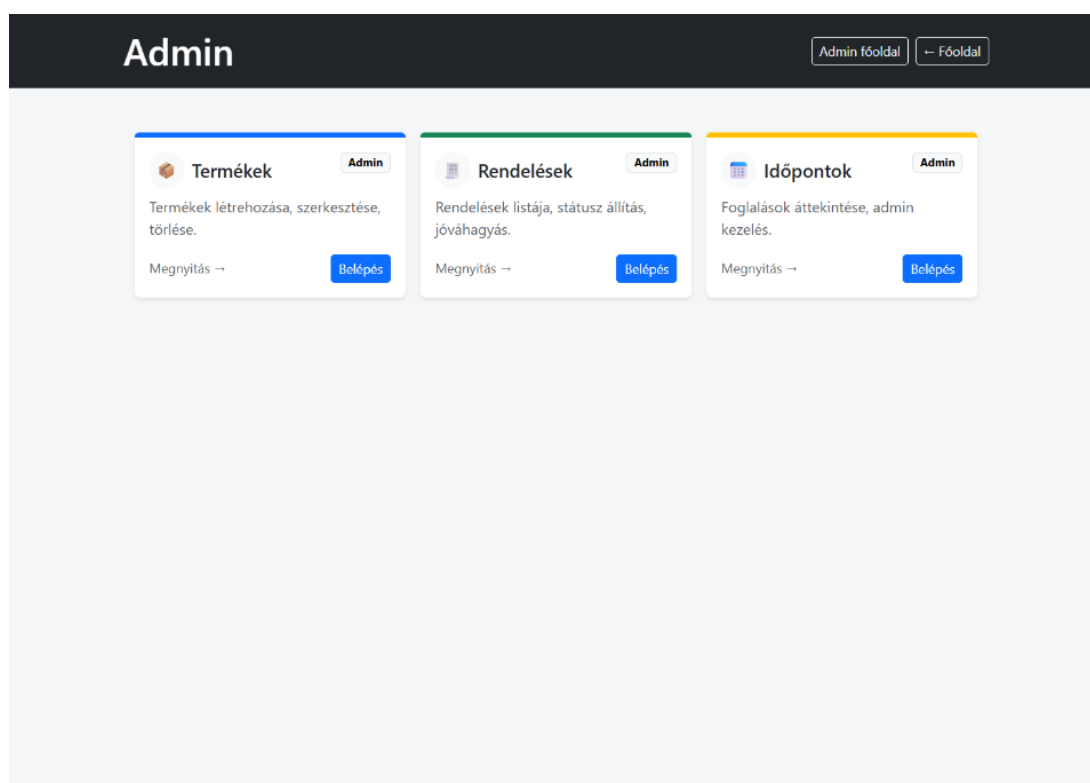
A rendszer adminisztrációs felülete külön jogosultsággal rendelkező felhasználók számára érhető el. Amennyiben egy felhasználó admin jogosultsággal jelentkezik be, a főoldal kezelősávjában egy **piros „Admin” gomb** jelenik meg, amelyen keresztül elérhető az adminisztrációs felület.

5.8.1. Admin oldal

Az admin felület célja a webshop és a foglalási rendszer működésének kezelése és felügyelete. Az adminisztrátor itt kezelheti a termékeket, megtekintheti és módosíthatja a rendeléseket, valamint áttekintheti az időpontfoglalásokat.

Az admin főoldalon három fő kezelőmodul érhető el:

- **Termékek kezelése** – új termékek létrehozása, meglévők szerkesztése és törlése
- **Rendelések kezelése** – a leadott rendelések listázása és státuszának módosítása
- **Időpontok kezelése** – a foglalások áttekintése és adminisztrációja



22. ábra – Az adminisztrációs felület főoldala

5.8.2. Termékek (Admin) oldal

Az admin felületen a termékek teljes körűen kezelhetők. Az adminisztrátor új termékeket hozhat létre, módosíthatja a meglévő termékek adatait, valamint törölheti azokat. A felület bal oldalán a már rögzített termékek listája látható, míg a jobb oldalon egy űrlap segítségével új termékek adhatók hozzá a rendszerhez. Az űrlapon megadható a termék cikkszáma, neve, ára, kategóriája, mértékegysége és termékcsoportja.

Admin Admin főoldal Főoldal

Termékek (Admin)

Létrehozás / szerkesztés / törlés + kategória, mértékegység, termékcsoport + Új termék

Lista

Lucfenyő gerenda 5m (WOOD-001) ID: 2 - Nettó: 10000 Ft - Faanyag - Mértékegység: db - Csoport: GEREENDA	Szerk	Töröl
Schmid RAPID SSF/WH 6,0 x 100/60 - 100db (SCHMID-RAPID-SSF-6x100-100) ID: 3 - Nettó: 5112 Ft - Szerkezetépítő csavarok - Mértékegység: db - Csoport: SCHMID_CSAVAR	Szerk	Töröl
Schmid RAPID SSF/WH 8,0 x 200/100 - 50db (SCHMID-RAPID-SSF-8x200-50) ID: 5 - Nettó: 8850 Ft - Szerkezetépítő csavarok - Mértékegység: db - Csoport: SCHMID_CSAVAR	Szerk	Töröl
Schmid RAPID SSF/WH 10,0 x 200/100 - 25db (SCHMID-RAPID-SSF-10x200-25) ID: 6 - Nettó: 7874 Ft - Szerkezetépítő csavarok - Mértékegység: db - Csoport: SCHMID_CSAVAR	Szerk	Töröl
RAPID süllyesztett feju szerkezeti facsavar (RAPID-SULLY-FEJ-SZERK) ID: 7 - Nettó: 10550 Ft - Facsavarok - Mértékegység: db - Csoport: RAPID_FACSAVAR	Szerk	Töröl
AEROSANA VISCONN fibre FEHÉR 600 ml (AEROSANA-VISCONN-FIBRE-600ML) ID: 8 - Nettó: 8000 Ft - Iomitok - Mértékegység: db - Csoport: AEROSAN_FIBRE	Szerk	Töröl
AEROSANA VISCONN fibre FEHÉR 5 L (AEROSANA-VISCONN-FIBRE-5L) ID: 9 - Nettó: 52000 Ft - Iomitok - Mértékegység: db - Csoport: AEROSAN_FIBRE	Szerk	Töröl
Pro Clima TESCON Primer RP 1 L (TESCON-PRIMER-RP-1L) ID: 10 - Nettó: 10000 Ft - Iomitok - Mértékegység: db - Csoport: TESCON_PRIMER	Szerk	Töröl
Pro Clima TESCON Primer RP 2.5 L (TESCON-PRIMER-RP-2_5L) ID: 11 - Nettó: 23000 Ft - Iomitok - Mértékegység: db - Csoport: TESCON_PRIMER	Szerk	Töröl
AEROSANA VISCONN légtömör fólia 600 ml (AEROSANA-VISCONN-600ML) ID: 12 - Nettó: 7000 Ft - Foliák - Mértékegység: db - Csoport: AEROSAN_MEMBRANE	Szerk	Töröl
AEROSANA VISCONN légtömör fólia 10 L (AEROSANA-VISCONN-10L) ID: 13 - Nettó: 99000 Ft - Foliák - Mértékegység: db - Csoport: AEROSAN_MLMBRANE	Szerk	Töröl
TESCON RAPIC 50 mm x 30 m (TESCON-RAPIC-50x30) ID: 14 - Nettó: 11000 Ft - Iomitok - Mértékegység: db - Csoport: TESCON_RAPIC	Szerk	Töröl
TESCON RAPIC 60 mm x 30 m (TESCON-RAPIC-60x30) ID: 15 - Nettó: 14000 Ft - Iomitok - Mértékegység: db - Csoport: TESCON_RAPIC	Szerk	Töröl
Lucfenyő gerenda 6m (WOOD-002) ID: 17 - Nettó: 12000 Ft - Faanyag - Mértékegység: db - Csoport: Gerenda	Szerk	Töröl
Lucfenyő gerenda 10 m (WOOD-003) ID: 20 - Nettó: 25000 Ft - Faanyag - Mértékegység: m - Csoport: Gerenda	Szerk	Töröl

Új termék

Cikkszám (ProductCode)
Pl. CS-6X120

Terméknév

Nettó ár (Ft)

Kategória
-- válassz --

Mértékegység
db

Termékcsoport
PI.GERENDA

Mentés

Megjegyzés: a cikkszám szerkesztésénél tilos van (UpdateInfo nem tartalmazza).

Termékek frissítése Kategóriák frissítése

23. ábra – Termékkezelő felület az adminisztrációs panelen

5.8.3. Rendelések (Admin) oldal

Az adminisztrátor ezen a felületen tekintheti meg a rendszerben leadott rendelések listáját. A táblázat tartalmazza a rendelés azonosítóját, a vásárló adatait, a rendelés dátumát, valamint a nettó, áfa és bruttó összeget. Az admin itt módosíthatja a rendelés státuszát, megtekintheti a rendelés tételeit, valamint szükség esetén törölheti a rendelést. A rendelések státusz alapján is szűrhetők.

Admin

Admin főoldal ← Főoldal

Rendelések (Admin)

Lista + státusz módosítás + tételek megtekintése

Frissítés

Szűrés státusz szerint:

Összes

	ID	Ügyfél	Email	Dátum	Nettó	ÁFA	Bruttó	Státusz	Megjegyzés	Művelet
+	18	Almássy Benjámín	almassy.benjamin@kkszki.hu	2026. 03. 11. 13:27:00	18550	5008.5	23558.5	Folya ▼ Tételek: 2	-	Törölés
+	17	Almássy Benjámín	almassy.benjamin@kkszki.hu	2026. 03. 07. 9:08:46	18550	5008.5	23558.5	Folya ▼ Tételek: 2	-	Törölés
+	16	Almássy Benjámín	almassy.benjamin@kkszki.hu	2026. 03. 07. 9:07:36	18000	4860	22860	Folya ▼ Tételek: 2	-	Törölés
+	15	Almássy Benjámín	almassy.benjamin@kkszki.hu	2026. 03. 07. 9:02:57	52200	14094	66294	Feldc ▼ Tételek: 2	-	Törölés

DB-ben angol státuszok vannak, a felületen magyarul jelenik meg.

24. ábra – Rendelések kezelése az admin felületen

5.8.4. Időpontfoglalások (Admin) oldal

Az adminisztrátor ezen a felületen tekintheti meg az ügyfelek által lefoglalt időpontokat egy naptárnézetben. A rendszer megjeleníti a foglalás időpontját és a foglaló felhasználó adatait. Az admin az egyes eseményekre kattintva jóváhagyhatja vagy lemondhatja a foglalást, így kezelve az időpontfoglalási rendszer működését.

Admin

Admin főoldal

Főoldal

Időpontok (Admin)

Naptár nézet + jóváhagyás/lemondás esemény kattintásra

2026. márc. 9 – 15.

today

<

>

	3. 9. H	3. 10. K	3. 11. Sze	3. 12. Cs	3. 13. P	3. 14. Szo	3. 15. V
08:00							
08:30				Jóváhagyva almassy.benjamin @kkszki.hu Sajó Péter- Tiszafüred			
09:00							
09:30							
10:00							
10:30							
11:00							
11:30							
12:00							
12:30							
13:00							
13:30							
14:00							
14:30							
15:00							
15:30							
16:00							
16:30							
17:00							
17:30							

Tipp: jelölj ki egy időszávot a naptárban a kézi felvitelhez. Kattintás eseményre: Pending → jóváhagyás, egyébként lemondás/infó.

25. ábra – Időpontfoglalások kezelése az admin felületen

6. Továbbfejlesztési lehetőségek

A rendszer jelenlegi formájában biztosítja a webshop működését, az időpontfoglalást, valamint a rendelésekkel kapcsolatos adminisztrációt. A jövőben azonban több további fejlesztési lehetőség is megvalósítható lenne.

Az egyik lehetséges fejlesztési irány egy **raktárkezelési modul** kialakítása, amely lehetővé tenné a termékek készlet szintjének nyilvántartását és automatikus frissítését a rendelések leadásakor.

További fejlesztési lehetőség lehet **online fizetési rendszer integrálása**, amely lehetővé tenné a bankkártyás vagy egyéb elektronikus fizetési módok használatát.

A rendszer később **számlázóprogrammal való integrációval** is bővíthető lenne. Ennek segítségével a rendelés leadása után automatikusan elkészülhetne a számla egy külső számlázórendszerben, amely jelentősen megkönnyítené az adminisztrációt.

A felhasználói élmény javítása érdekében megvalósítható lenne egy **fejlettebb keresési és szűrési rendszer**, valamint a termékekhez kapcsolódó képfeltöltési lehetőség is.

7. Összegzés

A dolgozat során egy olyan webes alkalmazás került megtervezésre és megvalósításra, amely támogatja egy kisvállalkozás webshop működését és az időpontfoglalási folyamat kezelését. A rendszer lehetővé teszi a termékek böngészését, a rendelések leadását, valamint a foglalások kezelését felhasználói és adminisztrátori oldalon is.

A fejlesztés során modern webes technológiák kerültek alkalmazásra, amelyek biztosítják a rendszer megbízható működését és bővíthetőségét. A kialakított architektúra lehetővé teszi a további fejlesztéseket, például raktárkezelési funkciók, online fizetés vagy számlázórendszer integrációjának megvalósítását.

A megvalósított rendszer gyakorlati példán keresztül mutatja be, hogyan lehet egy kisvállalkozás számára hasznos webes szolgáltatást létrehozni, amely támogatja a mindennapi működést és az ügyfelekkel való kapcsolattartást.

8. Tesztelési dokumentáció

8.1 Tesztelés célja

A projekt fejlesztése során fontos szempont volt a rendszer működésének ellenőrzése és a felhasználói felület helyes működésének biztosítása. Ennek érdekében kétféle tesztelési módszert alkalmaztunk: UI tesztelést és automatizált Selenium alapú tesztelést. A tesztelés célja annak biztosítása volt, hogy a rendszer fő funkciói megfelelően működnek, valamint a felhasználói felület elemei helyesen jelennek meg és használhatók.

8.2. UI Tesztelés

A UI (User Interface) tesztek a frontend komponensek működését vizsgálják. Ezek a tesztek ellenőrzik:

- az oldalak helyes betöltését
- az űrlapmezők megjelenítését
- a felhasználói input kezelését
- a gombok jelenlétét és működését

A tesztelés automatizált módon történt a következő eszközök segítségével:

- Bun test – JavaScript teszt futtatókörnyezet
- React Testing Library – React komponensek tesztelése
- Happy DOM – böngésző környezet szimulációja
- Jest-dom – DOM ellenőrző függvények

A UI tesztek előnyei:

- gyors futtatás
- könnyű hibakeresés fejlesztés közben
- jól reprodukálható eredmények

Tesztkörnyezet inicializálása

A tesztkörnyezet inicializálása a `setupTests.js` fájl segítségével történik. Ez a fájl létrehozza az szükséges DOM környezetet, valamint minden teszt után megtisztítja a tesztkörnyezetet.

```
Projekt-Frontend-main > src > JS setupTests.js > ...
1  import { afterEach } from "bun:test";
2  import { cleanup } from "@testing-library/react";
3  import { GlobalRegistrator } from "@happy-dom/global-registrator";
4  import "@testing-library/jest-dom";
5
6  GlobalRegistrator.register();
7
8  afterEach(() => {
9    cleanup();
10 });
11
12 console.log("Test DOM environment loaded");
```

26. ábra - `setupTests.js`

Login oldal UI tesztelése

A login oldal tesztelése során ellenőriztük, hogy a bejelentkezési felület megfelelően jelenik meg, valamint a felhasználó képes adatot bevinni a mezőkbe.

Teszt elemek:

- Login oldal megjelenítése – Az oldal címe megjelenik
- Email mező – A felhasználó email címet tud beírni
- Jelszó mező – A jelszó mező megjelenik
- Bejelentkezés gomb – A login gomb megjelenik

```
//login.test.jsx (részlet)
describe("Login UI teszt", () => {
  test("megjelenik a login oldal", () => {
    const { getByRole } = render(
      <MemoryRouter>
        <Login />
      </MemoryRouter>
    );

    expect(
      getByRole("heading", { name: /bejelentkezés/i })
    ).toBeInTheDocument();
  });
});
```

27. ábra - Login teszt kód

Regisztrációs oldal UI tesztelése

A regisztrációs oldal tesztelése során ellenőriztük az űrlapok összes mezőjének megjelenítését és működését.

Teszt elemek:

- Regisztrációs oldal – Az oldal megfelelően betölt
- Név mező – A felhasználó neve megadható
- Email mező – Email cím bevitel működik
- Telefonszám mező – Telefonszám mező megjelenik
- Számlázási cím mező – Cím mező megjelenik
- Jelszó mező – Jelszó mező megjelenik
- Jelszó újra mező – Jelszó megerősítő mező megjelenik
- Regisztrációs gomb – Regisztrációs gomb megjelenik

```
describe("Register UI teszt", () => {
  test("megjelenik a regisztrációs oldal", () => {
    const { getByRole } = render(
      <MemoryRouter>
        <Register />
      </MemoryRouter>
    );

    expect(
      getByRole("heading", { name: /regisztráció/i })
    ).toBeInTheDocument();
  });
});
```

28. ábra - Register teszt kód

Tesztek futtatása

A tesztek a Bun tesztfuttató segítségével indíthatók.

`cd.\Projekt-Frontend-main`

Be kell lépni a projekt mappába, ahonnan indítjuk a teszt parancsot.

`bun test --preload./src/setupTests.js`

Ez a parancs automatikusan lefuttatja az összes tesztet a `tests` mappában.

```
bun test v1.3.10 (30e609e0)

src\tests\login.test.jsx:
Test DOM environment loaded
✓ Login UI teszt > megjelenik a login oldal [31.00ms]
✓ Login UI teszt > megjelenik az email mező
✓ Login UI teszt > az email mezőbe lehet írni [16.00ms]
✓ Login UI teszt > megjelenik a jelszó mező
✓ Login UI teszt > megjelenik a bejelentkezés gomb

src\tests\register.test.jsx:
✓ Register UI teszt > megjelenik a regisztrációs oldal [15.00ms]
✓ Register UI teszt > megjelenik a név mező
✓ Register UI teszt > az email mezőbe lehet írni
✓ Register UI teszt > megjelenik a telefonszám mező [16.00ms]
✓ Register UI teszt > megjelenik a számlázási cím mező
✓ Register UI teszt > megjelenik a jelszó mező
✓ Register UI teszt > megjelenik a jelszó újra mező [16.00ms]
✓ Register UI teszt > megjelenik a regisztrációs gomb

13 pass
0 fail
13 expect() calls
Ran 13 tests across 2 files. [495.00ms]
```

28. ábra – Tesztek eredményei

8.3. Selenium alapú tesztelés (End-to-End)

A Selenium tesztek a rendszer teljes működését vizsgálják valós böngésző környezetben. Ezek az úgynevezett end-to-end tesztek a felhasználói folyamatokat modellezzik.

A tesztek Python nyelven készültek, Selenium WebDriver használatával. A ChromeDriver biztosítja a böngésző automatizálását.

A tesztek során az alábbi technológiák kerültek alkalmazásra:

- Selenium WebDriver – böngésző automatizálás
- ChromeDriver – Chrome böngésző vezérlése
- WebDriverWait, ExpectedConditions – dinamikus elemek kezelése
- ActionChains – egérműveletek szimulálása

A tesztelés során három kulcsfontosságú folyamat került ellenőrzésre:

- Bejelentkezés
- Termékrendelés (kosár használata)
- Időpontfoglalás

Bejelentkezés

A bejelentkezési folyamat ellenőrzése során a Selenium-teszt megnyitja a bejelentkezési oldalt, kitölti az email és jelszó mezőket, majd elküldi az űrlapot. A sikeres bejelentkezést követően a rendszer visszairányítja a felhasználót a főoldalra.

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5 import time
6 import os
7
8 Windurf: Refactor | Explain | Generate Docstring | X
9 def slow_type(element, text, delay=0.2):
10     for char in text:
11         element.send_keys(char)
12         time.sleep(delay)
13
14 Windurf: Refactor | Explain | Generate Docstring | X
15 def take_screenshot(driver, name):
16     folder = "login_screenshots"
17     os.makedirs(folder, exist_ok=True)
18     driver.save_screenshot(os.path.join(folder, f"{name}.jpg"))
19
20 Windurf: Refactor | Explain | Generate Docstring | X
21 def test_click_login_from_navbar():
22     driver = webdriver.Chrome()
23     driver.get("http://localhost:5173")
24
25     time.sleep(2)
26     take_screenshot(driver, "01_home_loaded")
27
28     time.sleep(3)
29
30     login_link = WebDriverWait(driver, 5).until(
31         EC.element_to_be_clickable((By.ID, "nav-login"))
32     )
33     login_link.click()
34
35     WebDriverWait(driver, 5).until(EC.url_contains("/login"))
36     time.sleep(1)
37     take_screenshot(driver, "02_login_page")
38
39     email_input = WebDriverWait(driver, 5).until(
40         EC.presence_of_element_located((By.CSS_SELECTOR, "input[type='email']"))
41     )
42     slow_type(email_input, "parmaandi.milangikszki.hu", delay=0.15)
43     take_screenshot(driver, "03_email_typed")
44
45     password_input = WebDriverWait(driver, 5).until(
46         EC.presence_of_element_located((By.CSS_SELECTOR, "input[type='password']"))
47     )
48     slow_type(password_input, "Dorimilan8991", delay=0.15)
49     take_screenshot(driver, "04_password_typed")
50
51     login_button = WebDriverWait(driver, 5).until(
52         EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']"))
53     )
```

29. ábra – Bejelentkezés teszt kódrészlet

Termékrendelés teszt

A termékrendelési teszt a webshop működését ellenőrzi. A teszt során a felhasználó bejelentkezik, megnyitja a terméklistát, kiválaszt egy terméket, kosárba helyezi, majd megnyitja a kosár oldalt. A teszt célja annak igazolása, hogy a kosár funkció megfelelően működik, és a kiválasztott termék megjelenik benne.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select, WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import os

Windsurf: Refactor | Explain | Generate Docstring | X
def slow_type(element, text, delay=0.2):
    for char in text:
        element.send_keys(char)
        time.sleep(delay)

Windsurf: Refactor | Explain | Generate Docstring | X
def take_screenshot(driver, name):
    folder = "order_screenshots"
    os.makedirs(folder, exist_ok=True)
    driver.save_screenshot(os.path.join(folder, f"{name}.jpg"))

Windsurf: Refactor | Explain | Generate Docstring | X
def test_place_order():
    driver = webdriver.Chrome()
    wait = WebDriverWait(driver, 10)
    driver.set_window_size(1920, 1080)

    driver.get("http://localhost:5173")
    wait.until(EC.presence_of_element_located((By.TAG_NAME, "body")))
    take_screenshot(driver, "01_home_loaded")

    # LOGIN
    driver.find_element(By.ID, "nav-login").click()

    email_input = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, 'input[type="email"]')))
    password_input = driver.find_element(By.CSS_SELECTOR, 'input[type="password"]')

    slow_type(email_input, "pazmandi.milan@kkszki.hu", 0.05)
    slow_type(password_input, "Dorimilan8991", 0.05)

    driver.find_element(By.CSS_SELECTOR, 'button[type="submit"]').click()
    time.sleep(2)
    take_screenshot(driver, "02_logged_in")

    # PRODUCTS
    driver.get("http://localhost:5173/products")
    wait.until(EC.presence_of_element_located((By.TAG_NAME, "body")))
    take_screenshot(driver, "03_products_page")
```

27. ábra – Rendelés teszt kódrészlet

Időpontfoglalás teszt

Az időpontfoglalási teszt a FullCalendar alapú naptár működését ellenőrzi. A felhasználó bejelentkezik, megnyitja az időpontfoglalási oldalt, kijelöl egy időintervallumot (például 10:00–12:00), megjegyzést ír, majd elküldi a foglalást. A teszt vizuálisan is ellenőrzi a kijelölt időpont megjelenését.

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 Click to collapse the range. iver.common.action_chains import ActionChains
4 import time
5 import os
6
7 def slow_type(element, text, delay=0.20):
8     for char in text:
9         element.send_keys(char)
10        time.sleep(delay)
11
12 def take_screenshot(driver, name):
13     folder = "appointment_screenshots"
14     os.makedirs(folder, exist_ok=True)
15     driver.save_screenshot(os.path.join(folder, f"{name}.jpg"))
16
17 def test_fixed_appointment():
18     driver = webdriver.Chrome()
19     driver.set_window_size(1920, 1080)
20
21     # --- LOGIN FOLYAMAT ---
22     driver.get("http://localhost:5173/appointments")
23     time.sleep(2)
24
25     driver.find_element(By.LINK_TEXT, "Bejelentkezés").click()
26     time.sleep(2)
27
28     slow_type(driver.find_element(By.CSS_SELECTOR, 'input[type="email"]'),
29              "pazmandi.milan@kkszi.hu", 0.15)
30     slow_type(driver.find_element(By.CSS_SELECTOR, 'input[type="password"]'),
31              "Dorimilan8991", 0.15)
32
33     driver.find_element(By.CSS_SELECTOR, 'button[type="submit"]').click()
34     time.sleep(3)
35
36     driver.get("http://localhost:5173/appointments")
37     time.sleep(3)
38     take_screenshot(driver, "calendar_loaded")
39
40     # --- 1) 10:00 és 12:00 LABEL MEGKERESÉSE ---
41     labels = driver.find_elements(By.CSS_SELECTOR, ".fc-timegrid-slot-label")
42
43     ten_label = None
44     twelve_label = None
```

28. ábra – Időpontfoglalás teszt kódrészlet

Selenium tesztek futtatása

A Selenium alapú tesztek Python nyelven készültek, és Visual Studio Code fejlesztői környezetben kerültek futtatásra.

A tesztek futtatásához szükséges a megfelelő környezet beállítása, amely tartalmazza a Python értelmezőt, a Selenium csomagot, valamint a ChromeDriver alkalmazást. A ChromeDriver biztosítja a kapcsolatot a Selenium és a Google Chrome böngésző között.

A tesztek indítása parancssorból történik a projekt mappájában. A parancs végrehajtása után a Chrome böngésző automatikusan elindul, és a teszt lépésről lépésre végrehajtja a felhasználói műveleteket (pl. bejelentkezés, űrlapkitöltés, gombkattintás).

A teszt futása közben a Selenium a megadott utasítások alapján vezérli a böngészőt, és ellenőrzi az egyes lépések eredményét. A dinamikusan betöltődő elemek kezelésére a WebDriverWait és az ExpectedConditions kerül alkalmazásra, amely biztosítja, hogy a teszt csak akkor lépjen tovább, amikor az adott elem már elérhető.

A tesztek futása során képernyőképek is készíthetők, amelyek a dokumentációban bemutatják a sikeres működést.

Összegzés

A kétféle tesztelési módszer egymást kiegészítve biztosítja a rendszer megbízható működését.

A UI tesztek gyors visszajelzést adnak az egyes felületi elemek működéséről, míg a Selenium tesztek a teljes felhasználói folyamatok helyes működését ellenőrzik valós környezetben.

9. Irodalomjegyzék

[1]

Téma: ASP.NET Core webalkalmazás-fejlesztési keretrendszer

Webcím: <https://learn.microsoft.com/aspnet/core>

Letöltés dátuma: 2026.03.10.

[2]

Téma: Entity Framework Core adatbázis-kezelő keretrendszer

Webcím: <https://learn.microsoft.com/ef/core>

Letöltés dátuma: 2026.03.10.

[3]

Téma: JSON Web Token hitelesítési szabvány

Webcím: <https://datatracker.ietf.org/doc/html/rfc7519>

Letöltés dátuma: 2026.03.10.

[4]

Téma: React JavaScript könyvtár dokumentációja

Webcím: <https://react.dev>

Letöltés dátuma: 2026.03.10.

[5]

Téma: OpenAPI specifikáció és API dokumentáció

Webcím: <https://www.openapis.org>

Letöltés dátuma: 2026.03.10.

[6]

Téma: Swagger API dokumentációs eszköz

Webcím: <https://swagger.io>

Letöltés dátuma: 2026.03.10.